

Deploying Challenge Copilot on Hugging Face Spaces

LlamaIndex + OpenAI + Gradio (RAG over challenge_context.txt)

Quick Summary

- You will deploy a Gradio app as a Hugging Face Space.
- Your knowledge base lives in challenge_context.txt inside the Space repo.
- You will add OPENAI_API_KEY as a Secret in Space Settings (never hardcode keys).
- The Space needs two core files: app.py and requirements.txt (plus challenge_context.txt).

What you are building

A minimal Retrieval-Augmented Generation (RAG) chatbot that answers questions about the Building AI Application Challenge using a single text file as its knowledge base. At runtime it retrieves the most relevant chunks from the text file (LlamaIndex VectorStoreIndex) and generates a grounded response using the OpenAI API. The user interface is a Gradio chat.

Prerequisites

- A Hugging Face account.
- An OpenAI API key.
- Your official challenge content saved into a text file: challenge_context.txt.
- Basic familiarity with GitHub-style repos (uploading files to a Space).

Folder and file structure in the Space

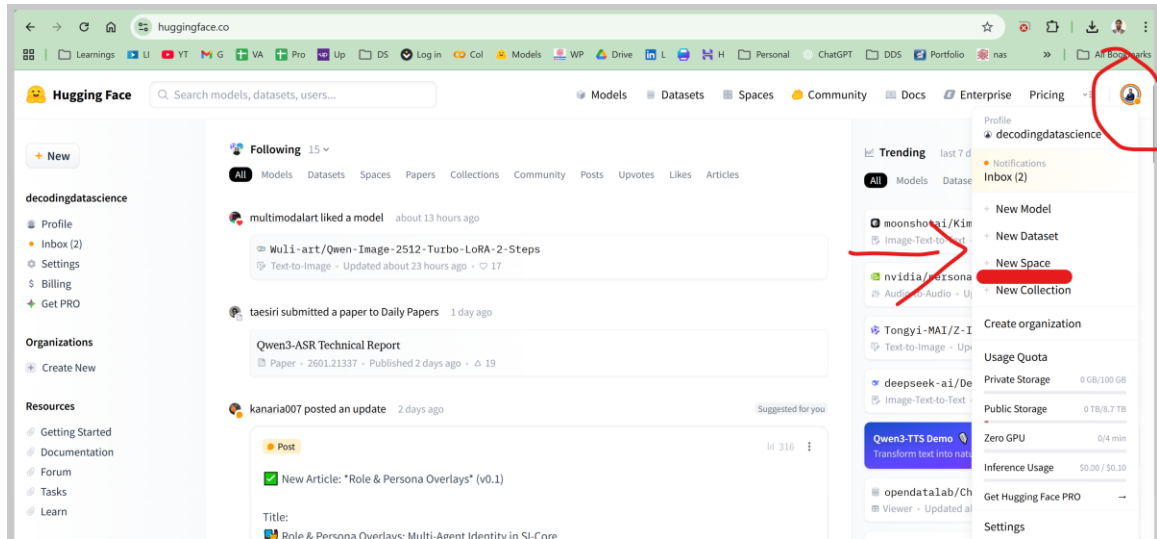
Your Space repository should contain:

- app.py (the Gradio + LlamaIndex app)
- requirements.txt (Python dependencies for the Space build)
- challenge_context.txt (the knowledge base text file)

Step-by-step deployment (Hugging Face Spaces)

1. Create a new Space

- Go to Hugging Face and create a new Space.



- Choose SDK: Gradio.
- Pick a Space name (e.g., challenge-copilot).



Create a new Space

Spaces are Git repositories that host application code for Machine Learning demos.
You can build Spaces with Python libraries like Gradio, or using Docker images.

Owner	Space name
decodingdatascience ▾	New Space name
Short description	
Short Description	
License	
License	

Select the Space SDK

You can choose between Gradio, Docker, or Static to host your Space.

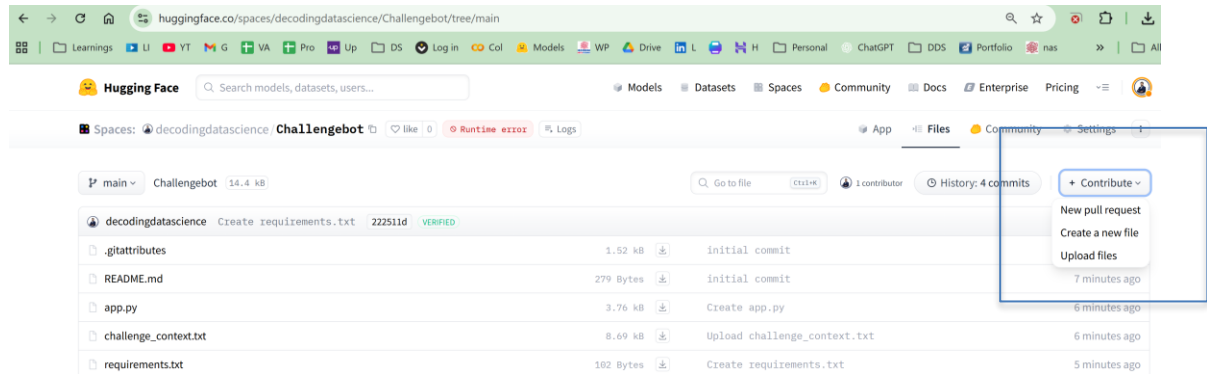
 Gradio 4 templates	 Docker 17 templates	 Static 6 templates
--	---	--

Choose a Gradio template:

-------------	-------------	-------------	-------------

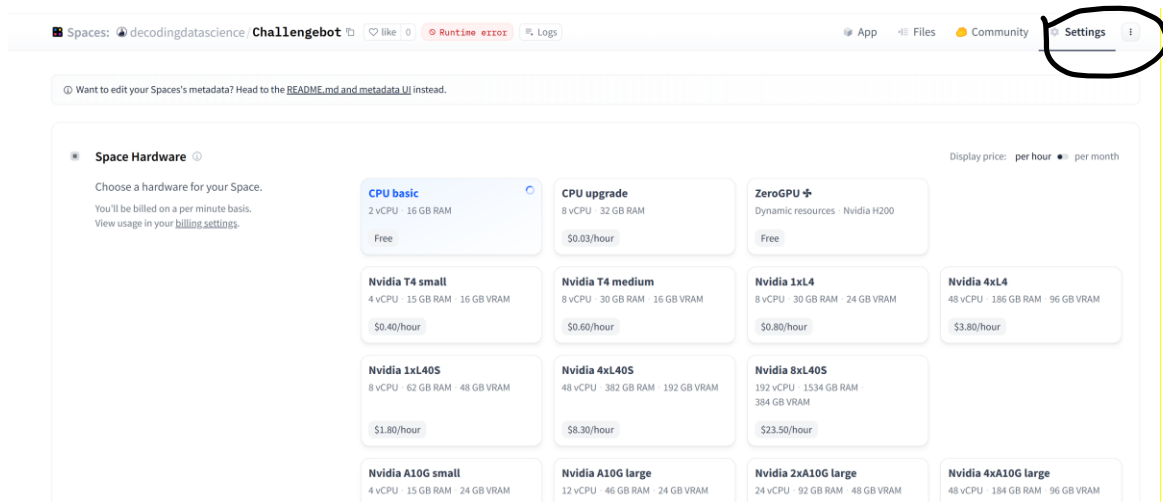
2. Add your three files to the Space repo

- Upload/commit app.py and requirements.txt (provided in Appendix A and B).
- Add challenge_context.txt to the repo root (same folder as app.py).
- If you update the text file later, just commit the new version and the Space will rebuild/restart.



3. Add the OpenAI key as a Secret

- Open your Space → Settings → Variables and secrets.
- Add a Secret named OPENAI_API_KEY and paste your OpenAI key.
- Do not put keys inside app.py, requirements.txt, or challenge_context.txt.



4. Wait for the build and start the app

- After files are committed, Hugging Face installs dependencies from requirements.txt.
- When the build finishes, the Space should switch to 'Running'.
- Open the Space URL; you should see the Gradio chat UI.

5. Test the RAG behavior

- Ask: 'What will I build in this live session?'
- Ask: 'Who is this best for?'
- Ask something not in the file; the bot should say it does not know and suggest adding more official content.

Optional configuration (environment variables)

You can define these in Space Settings → Variables (not secrets) to tune behavior:

Variable	Purpose
OPENAI_MODEL	Which chat model to use (default: gpt-4o-mini).
OPENAI_EMBED_MODEL	Which embedding model to use (default: text-embedding-3-small).
TOP_K	How many chunks to retrieve per question (default: 3).
SHOW_SOURCES	true/false to append sources (default: true).
DOC_PATH	Path to your context file (default: challenge_context.txt).

Updating the knowledge base (challenge_context.txt)

1. Open challenge_context.txt in the Space repo.
2. Paste the latest official challenge content (overview, tracks, checkpoints, rules, submission format, prizes).
3. Commit changes. The Space will rebuild/restart automatically.
4. Re-test by asking questions about the newly added sections.

Troubleshooting

Build fails during dependency install

- Check requirements.txt spelling and version pins.
- Open the Space build logs to see the exact error line.

- Start with the minimal requirements provided here; add extra libraries only when needed.

Runtime error: OPENAI_API_KEY missing

- Add OPENAI_API_KEY in Space Settings → Variables and secrets as a Secret.
- Restart the Space after adding the secret.

Answers are too generic or missing details

- Your challenge_context.txt likely does not contain enough official text.
- Add more structured content with headings (Tracks, Checkpoints, Rules, Prizes).
- Increase TOP_K to 5 and try again.

High latency or rate limits

- Use a smaller model (e.g., keep gpt-4o-mini).
- Reduce TOP_K and chunk size if needed.
- Consider caching and persisting the index for larger corpora (advanced).

Appendix A — app.py

```
import os
from pathlib import Path
import gradio as gr

from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding

MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
TOP_K = int(os.getenv("TOP_K", "3"))
DOC_PATH = Path(os.getenv("DOC_PATH", "challenge_context.txt"))

SYSTEM_GUARDRAILS = (
    "You are Challenge Copilot. Answer ONLY using the provided context. "
    "If the answer is not in the context, say: 'I don't know based on the current "
    "document.' "
    "Then ask the user to add the missing official details to challenge_context.txt."
)

_INDEX = None
_QUERY_ENGINE = None

def build_index():
    global _INDEX, _QUERY_ENGINE
    if _QUERY_ENGINE is not None:
```



```

    return _QUERY_ENGINE

    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise RuntimeError(
            "OPENAI_API_KEY is missing. Add it in the Space Settings → Variables and secrets."
        )

    if not DOC_PATH.exists():
        DOC_PATH.write_text("Add the official challenge content here.\n",
            encoding="utf-8")

    Settings.llm = OpenAI(model=MODEL, temperature=0.2)
    Settings.embed_model = OpenAIEmbedding(model=EMBED_MODEL)
    Settings.chunk_size = 800
    Settings.chunk_overlap = 120

    data_dir = str(DOC_PATH.parent)
    docs = SimpleDirectoryReader(
        input_dir=data_dir,
        required_exts=[".txt"],
        recursive=False
    ).load_data()

    docs = [d for d in docs if d.metadata.get("file_name") == DOC_PATH.name]
    if not docs:
        raise FileNotFoundError(f"Could not load {DOC_PATH.name}. Make sure it exists in the repo.")

    _INDEX = VectorStoreIndex.from_documents(docs)
    _QUERY_ENGINE = _INDEX.as_query_engine(similarity_top_k=TOP_K)
    return _QUERY_ENGINE

def format_sources(resp, max_sources=3, max_chars=220):
    lines = []
    for i, sn in enumerate(getattr(resp, "source_nodes", [])[:max_sources], start=1):
        fn = sn.node.metadata.get("file_name", "unknown")
        snippet = sn.node.get_content().replace("\n", " ").strip()[:max_chars]
        score = getattr(sn, "score", None)
        score_txt = f" (score={score:.3f})" if isinstance(score, (float, int)) else ""
        lines.append(f"{i}. {fn}{score_txt}: {snippet}...")
    return "\n".join(lines) if lines else "No sources returned."

def chat(message, history):
    qe = build_index()
    prompt = (
        f"{SYSTEM_GUARDRAILS}\n\n"
        f"User question: {message}\n"
        f"Answer using ONLY the context."
    )

```

```

resp = qe.query(prompt)
answer = str(resp).strip()

show_sources = os.getenv("SHOW_SOURCES", "true").lower() == "true"
if show_sources:
    answer += "\n\n---\nSources:\n" + format_sources(resp, max_sources=TOP_K)
return answer

demo = gr.ChatInterface(
    fn=chat,
    title="Challenge Copilot – RAG Q&A Bot",
    description="Ask questions about the challenge using challenge_context.txt (LlamaIndex + OpenAI).",
    examples=[
        "What will I build in this live session?",
        "Who is this best for?",
        "What are the prerequisites?"
    ],
    theme="soft"
)

if __name__ == "__main__":
    demo.launch()

```

Appendix B — requirements.txt

```

gradio>=4.0.0
llama-index>=0.10.0
llama-index-llms-openai>=0.1.0
llama-index-embeddings-openai>=0.1.0

```

Appendix C — challenge_context.txt (suggested structure)

Building AI Application Challenge – Official Content

Overview

- What the challenge is
- Who it is for
- Key outcomes

Tracks

- 1) LLM / API Integration Path
- 2) No-code / Low-code Path
- 3) (Any additional path you support)

Checkpoints

Day 1:

Day 2:



...

Day 8:

Submission Format

- What to submit
- Links required (GitHub, Hugging Face Space, demo URL)

Prizes

- Scholarships, credits, licenses

FAQs

- Common questions and answers